

VERSION 1.0

DATE 04/10/2026



SPECIFICATIONS TECHNIQUES

LES INSTRUCTEURS DU CIEL

ÉMIS PAR : SAULAS ENZO
NOM DE L'ENTREPRISE

Page laissée vide volontairement

TABLE DES MATIERES

1. Introduction	3
1.1 Contexte du projet	3
1.2 Objectif de l'application	3
2. Architecture générale	3
2.2 Description de l'architecture	3
2.2 Architecture logicielle	3
2.3 ARCHITECTURE MATERIELLE	3
2.4 Contraintes techniques	3
3. Spécifications fonctionnelles	4
3.1 Cas d'utilisations	4
3.2 Diagrammes d'activités	4
4. Spécifications Techniques backend.....	5
4.1 Langages et framework	5
4.2 structure du projet maven	5
4.3 API REST	5
4.4 Services	5
4.5 Repository	5
4.6 Sécurité	5
4.7 Configuration Spring	5
5. Base de Données.....	6
5.1 Contraintes générales	6
5.2 Modélisation	6
5.2.1 Dépendances fonctionnelles	6
5.2.2 Modèle conceptuel de données	6
5.2.3 Modele logique de données	6
5.2.4 modele physique de données.....	6
5.3 Script sql.....	6
5.4 Procédures stockées	6
5.5 Fonctions SQL.....	6
5.6 Triggers.....	6
5.7 Politique de sauvegarde et de restauration.....	6
5.8 Outils utilisés	6
6. Spécifications tehcniques frontend	7

6.1 Architecture du front	7
6.2 Pages / composants	7
6.3 Appels API	7
6.4 Gestions des etats.....	7
6.5 Sécurité Front.....	7
7. Outils de développement	8
7.1 IDE	8
7.2 Gestions des versions.....	8
7.3 Ci/CD	8
7.4 Conteneurisation	8
7.5 Documentation	8
8. Tests	9
8.1 Stratégie de tests	9
8.2 Tests unitaires	9
8.3 tests d'integrations	9
8.4 Tests API	9
8.5 Converture de testS	9
8.6 Gestions des donnees de tests	9
9. Sécurité	10
9.1 Normes et bonnes pratiques	10
9.2 Protection des données sensibles	10
9.3 Gestion des mots de passe	10
9.4 Chiffrement	10
9.5 Protection contre les attaques (OWASP)	10
10. Déploiement	11
10.1 Environnements (dev, test, staging, prod)	11
10.2 Pipeline CI/CD10.3 Packaging (Jar, Docker image).....	11
10.4 Configuration serveur (Tomcat, Nginx, Apache)	11
10.5 Monitoring	11
11. Annexes.....	12
11.1 Diagrammes UML supplémentaires	12
11.2 Exemples de requêtes SQL.....	12
11.3 Exemples de payloads API	12
11.4 Glossaire technique	12

1. INTRODUCTION

1.1 CONTEXTE DU PROJET

Ce projet est un projet personnel réalisé dans une démarche d'apprentissage du développement d'applications fullstack. Il ne répond à aucune commande d'un client : son but est de me permettre de mettre en pratique, sur un cas concret, l'ensemble des étapes de conception et de réalisation d'une application web, de la modélisation des données jusqu'à l'interface utilisateur.

Le sujet choisi est la gestion d'une école de pilotage : des apprentis pilotes réservent des vols encadrés par des instructeurs, sur des avions entretenus par des techniciens, en suivant un parcours de formations successives. Ce domaine a l'avantage de comporter de nombreuses règles métier (disponibilité des avions, validité des contrôles techniques, correspondance entre formation, instructeur et appareil), ce qui offre un terrain réaliste pour travailler la conception d'une base de données et la logique applicative. La modélisation s'appuie sur la méthode Merise (MCD, puis MLD), et l'application est développée avec Java, Spring Boot et une base de données MariaDB.

1.2 OBJECTIF DE L'APPLICATION

L'objectif de l'application est de permettre la gestion centralisée des vols d'une école de pilotage : planifier et réserver des vols, suivre les formations de chaque apprenti, affecter un instructeur et un avion compatibles, et suivre l'état technique de la flotte grâce aux contrôles techniques. L'application doit garantir le respect des règles métier, par exemple en refusant un vol si l'avion n'est pas disponible, si son contrôle technique n'est pas validé ou si l'instructeur ne correspond pas à la formation de l'apprenti.

L'objectif pédagogique est plus important que l'objectif fonctionnel. À travers ce projet, je cherche à apprendre à :

- modéliser une base de données à partir de dépendances fonctionnelles et de contraintes ;
- construire une API et une logique applicative avec Spring Boot ;
- relier cette partie serveur à une interface utilisateur ;
- gérer un projet avec Git et GitHub, en séparant ce qui doit être versionné de ce qui doit rester privé ;
- documenter mon travail de façon claire.

2. ARCHITECTURE GENERALE

2.2 DESCRIPTION DE L'ARCHITECTURE

Schéma global

A venir

2.2 ARCHITECTURE LOGICIELLE

Spring boot, modules, couches

A venir

2.3 ARCHITECTURE MATERIELLE

Serveurs, hébergement, environnement

A venir

2.4 CONTRAINTES TECHNIQUES

Versions, normes, performances, sécurité

3. SPECIFICATIONS FONCTIONNELLES

3.1 CAS D'UTILISATIONS

A venir

3.2 DIAGRAMMES D'ACTIVITES

A venir

4. SPECIFICATIONS TECHNIQUES BACKEND

4.1 LANGAGES ET FRAMEWORK

- Java 25
- Spring Boot
- Spring Web
- JPA / Hibernate
- MariaDB JDBC

4.2 STRUCTURE DU PROJET MAVEN

- modules
- organisation des packages

4.3 API REST

- Endpoints
- méthodes http
- DTO / Payload
- Codes de retour
- Gestions des erreurs

4.4 SERVICES

- Logique métier
- Interfaces et implémentations

4.5 REPOSITORY

- Interface JPA
- Requêtes personnalisées

4.6 SECURITE

- Authentification (JWT, Oauth2 ...)
- Autorisation (rôles et permissions)
- Gestions des sessions

4.7 CONFIGURATION SPRING

- application.yml
- Profiles (dec, test, prod)

- Logging

5. BASE DE DONNEES

5.1 CONTRAINTES GENERALES

- Performances
- Intégrité
- Sécurité

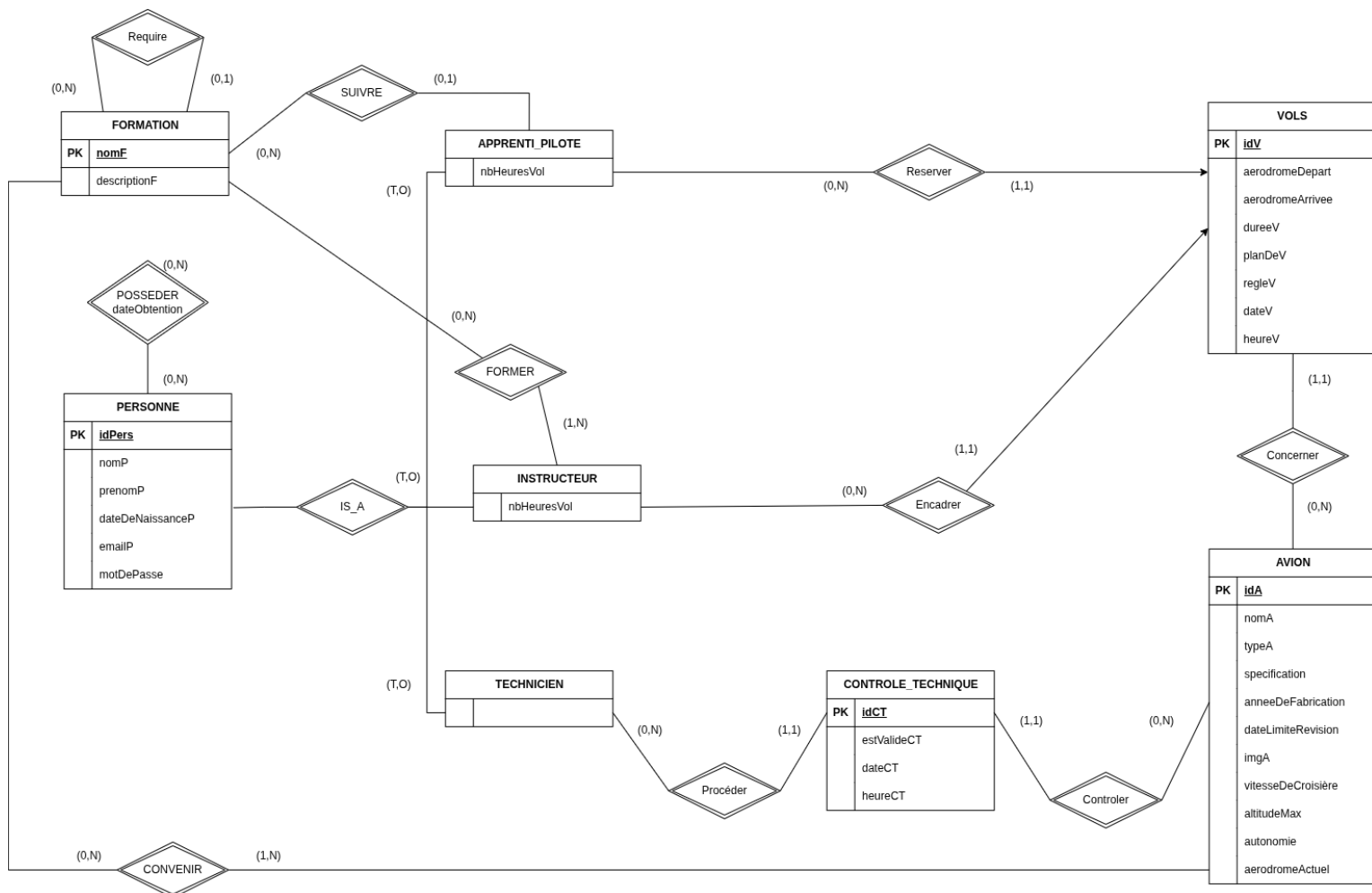
5.2 MODELISATION

5.2.1 DEPENDANCES FONCTIONNELLES & CONTRAINTES

- DF1: idPers -> nomP, prenomP, dateDeNaissance, emailP, motdePasse
- DF 2: idAppP -> nbHeuresDeVol, nomF
- DF 3: idIns -> nbHeuresDeVol
- DF 4: nomF -> descriptionF, formationNecessaire
- DF 5: idPers, nomF -> dateObtention
- DF 6: idV -> aerodromeDepart, aerodromeArrivee, dureeV, planDeV, regleV, dateV, heureV, idAppP, idIns, idA
- DF 7: idAppP, dateV, heureV -> idV idA, idIns
- DF 8: idIns, dateV, heureV -> idV, idA, idAppP
- DF9: idA -> nomA, typeA, specification, anneeDeFabrication, dateLimite, imgA, vitesseDeCroisiere, altitudeMax, aerodromeActuel
- DF 10: idCT -> estValideCT, dateCT, heureCT, idA, idTech
- DF 11: idTech, dateCT -> idCT, idA

- C1 : Un avion ne peut pas être sur deux vols qui se chevauchent dans le temps.
- C2 : aerodromeDepart d'un vol doit être égal à aerodromeActuel de l'avion, une fois le vol terminé, aerodromeActuel prend automatiquement la valeur de aerodromeArrivee.
- C3 : Un contrôle technique ne peut avoir lieu que si l'avion n'est dans aucun vol à ce moment là.
- C4 : dureeV doit être strictement inférieur à l'autonomie de l'avion.
- C5 : Un avion ne peut décoller que si son dernier contrôle technique est valide.
- C6 : Un avion ne peut pas décoller si dateV est postérieure à dateLimiteRevision.
- C7 : un instructeur ne peut encadrer un vol si la formation en cours de l'apprenti fait partie des formations qu'il enseigne.
- C8 : un apprenti ne peut voler que sur un avion lié à sa formation en cours.
- C9 : un instructeur à au moins 200 h de vol
- C 10: Une personne ne peut suivre ou obtenir une formation que si la formation requise figure déjà dans ses formations obtenues. Cela vaut pour les deux chaînes :
pilotes : BIA -> ABL -> LAPL -> PPL -> CPL -> ATPL -> MPL
instructeurs : CAEA -> IR -> QT -> MCC
- C11 : Un apprenti ou un instructeur ne peut pas être sur deux

5.2.2 MODELE CONCEPTUEL DE DONNEES



5.2.3 MODELE LOGIQUE DE DONNEES

PERSONNE [idPers, nomP, prenomP, dateDeNaissanceP, emailP, motDePasse]

FORMATION [nomF, descriptionF, formationNecessaire]

APPRENTI_PILOTE [#idAppP, nbHeuresVol, #nomF]

INSTRUCTEUR [#idIns, nbHeuresVol]

TECHNICIEN [#idTech]

AVION [idA, nomA, typeA, specification, anneeDeFabrication, dateLimiteRevision, imgA, vitesseDeCroisiere, altitudeMax, aerodromeActuel]

VOLS [idV, aerodromeDepart, aerodromeArrivee, dureeV, planDeV, regleV, dateV, heureV, #idAppP, #idIns, #idA]

CONTROLE_TECHNIQUE [idCT, estValideCT, dateCT, heureCT, #idTech, #idA]

POSSEDER [#idPers, #nomF, dateObtention]

FORMER [#idIns, #nomF]

CONVENIR [#idA, #nomF]

5.2.4 MODELE PHYSIQUE DE DONNEES

5.3 SCRIPT SQL

- création de tables
- index
- Contraintes(PK, FK, uniques, check)

5.4 PROCEDURES STOCKEES

A venir

5.5 FONCTIONS SQL

A venir

5.6 TRIGGERS

A venir

5.7 POLITIQUE DE SAUVEGARDE ET DE RESTAURATION

A venir

5.8 OUTILS UTILISES

- JetBrains DataGrip
- MariaDB CLI / Admin tools

6. SPECIFICATIONS TECHNIQUES FRONTEND

6.1 ARCHITECTURE DU FRONT

A venir

6.2 PAGES / COMPOSANTS

A venir

6.3 APPELS API

A venir

6.4 GESTIONS DES ETATS

A venir

6.5 SECURITE FRONT

A venir

7. OUTILS DE DEVELOPPEMENT

7.1 IDE

- IntelliJ IDEA
- DataGrip

7.2 GESTIONS DES VERSIONS

- git
- conventions de branches

7.3 CI/CD

- GitHub actions

7.4 CONTENEURISATION

- docker

7.5 DOCUMENTATION

- javadoc
- postman collections

8. TESTS

8.1 STRATEGIE DE TESTS

A venir

8.2 TESTS UNITAIRES

- JUnit 5
- mockito
- tests de services
- tests des repositories

8.3 TESTS D'INTEGRATIONS

- Spring boot test
- Testcontainers (MariaDB)

8.4 TESTS API

- RestAssured

8.5 CONVERTURE DE TESTS

- JaCoCo

8.6 GESTIONS DES DONNEES DE TESTS

9. SECURITE

9.1 NORMES ET BONNES PRATIQUES

9.2 PROTECTION DES DONNEES SENSIBLES

9.3 GESTION DES MOTS DE PASSE

9.4 CHIFFREMENT

9.5 PROTECTION CONTRE LES ATTAQUES (OWASP)

- XSS
- CSRF
- Injection SQL
- Bruteforce
- Session hijacking

10. DEPLOIEMENT

10.1 ENVIRONNEMENTS (DEV, TEST, STAGING, PROD)

10.2 PIPELINE CI/CD10.3 PACKAGING (JAR, DOCKER IMAGE)

10.4 CONFIGURATION SERVEUR (TOMCAT, NGINX, APACHE)

10.5 MONITORING

- Logs
- Metrics (Prometheus, Grafana)

11. ANNEXES

11.1 DIAGRAMMES UML SUPPLEMENTAIRES

11.2 EXEMPLES DE REQUETES SQL

11.3 EXEMPLES DE PAYLOADS API

11.4 GLOSSAIRE TECHNIQUE